PROJECT-BASED LEARNING

PHASE-II EVALUATION

Inverted Index and Code Based Search Engine

Team ID: DSCPP-III-2026-T112

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T112
Semester: 3rd
Section: A, B, C, F
Domain: CSE

Team Members

1. Anmol Gupta – 2510385219
2. Harshit Kumar – 2510012451
3. Atharva Goyal – 2510360047
4. Navneet Nainwal – 2510010627

Mentor

(Prof.) Dr. Jyoti Agarwal

Interactions completed: ___ Yes ___

Phase-I Recap & Feedback Incorporated

Phase-I Status

During Phase-I, we proposed an Inverted Index-based search system to improve the efficiency of searching for keywords across multiple documents. The basic project design, objectives, data structures, and implementation approach were identified.

Feedback Received

- 1. Improve the clarity of the project scope and problem statement.
- 2. Explain the data structures used for indexing and searching.
- 3. Include a clear testing and validation strategy.

Changes Incorporated

- 1. Refined the problem statement and defined the project scope.
- 2. Identified suitable data structures (Hash Tables & Linked Lists).
- 3. Planned comprehensive unit tests for tokenization, Boolean queries, and memory.

Problem Statement & Finalized Objectives

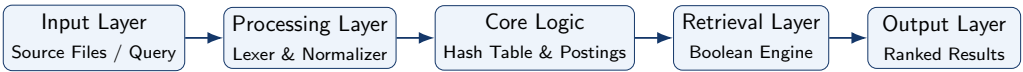
Problem Statement

Searching for specific keywords across multiple code files using naive sequential scanning becomes highly inefficient ($O(N)$) as the codebase scales. The project aims to develop an in-memory Inverted Index retrieval engine with Boolean search capabilities for fast, structured retrieval.

Finalized Objectives

1. **Implement Inverted Index:** Build a hash-table index mapping unique tokens to posting lists.
2. **Document Preprocessing:** Lexical analysis of source files with normalization and comment stripping.
3. **Boolean Retrieval Engine:** Support linear $O(M + N)$ two-pointer AND, OR, and NOT evaluation.
4. **Testing & Memory Safety:** Validate retrieval correctness with zero heap memory leaks.

Updated System Architecture / Workflow



Major Components

- Lexical Tokenizer & Normalization Engine
- In-Memory Inverted Index (Hash Table + Linked Lists)
- Query Parser & Boolean Operations (AND, OR, NOT)
- Relevance Ranking Engine (TF-IDF & Snippets)

Changes Since Phase-I

- Adopted hybrid C/C++ architecture via `extern "C"`
- Upgraded to linear $O(M + N)$ two-pointer list intersection
- Added explicit heap memory cleanup to prevent leaks
- Modular header decoupling (`index.h`, `query_parser.h`)

Module Design & Functional Requirements

Major Modules

1. Ingestion Pipeline & Lexical Tokenizer (C++) — Reads source files, removes comments, normalizes text.
2. In-Memory Inverted Index Engine (C) — Hash table using djb2 and linked-list postings.
3. Query Parsing & Boolean Operations (C/C++) — Parses queries and executes AND, OR, and NOT.
4. Relevance Ranking & CLI (C/C++) — Calculates TF-IDF and displays ranked outputs.

Functional Requirements

- 1. Scan source files recursively and assign document IDs.
- 2. Build in-memory index with term frequencies.
- 3. Support multi-word Boolean search queries.
- 4. Rank results via TF-IDF and show matched snippets.

Module Dependency

Module 1 feeds clean tokens into Module 2 to build the in-memory index. Module 3 evaluates queries by requesting posting lists from Module 2, and forwards matching documents to Module 4 for TF-IDF ranking.

Technology Stack & Development Methodology

Technology Stack

- **Programming:** C (Core Indexing) & C++17 (Parsing, STL)
- **Data Structures:** Hash Table, Singly Linked Lists, Min-Heap
- **Compiler:** GCC / Clang (`g++ -std=c++17`)
- **Development Tool:** Visual Studio Code, Make
- **Version Control:** Git & GitHub

Development Methodology

1. Requirement Analysis & Interface Definition
2. Modular Architecture Design (`include/` & `src/`)
3. Core Module Implementation (C/C++ Hybrid)
4. Integration Testing with Unit Harnesses
5. Benchmark Evaluation & Profiling

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	Hash Tables (djb2 hash, separate chaining), Singly Linked Lists (Postings), Dynamic Arrays	Module 2 & 3
OOPs with C++	Encapsulation, RAII, STL (`std::vector`, `std::string`, `std::stringstream`)	Module 1 & 4
Operating Systems	File I/O Streams, Heap Memory Management (`malloc`/`free`), Process execution overhead	Module 1 & Core
Design & Analysis of Algorithms	Two-pointer sorted list intersection ($O(M+N)$), Top-K selection using Min-Heap ($O(N \log K)$)	Module 3 & 4

Important: Mention only those concepts that have been **actually implemented** in the project.

Development & Implementation Progress

Module	Status	Work Completed
Module 1: Ingestion & Tokenizer	Completed	File reading, token extraction, case normalization, punctuation filtering.
Module 2: Inverted Index Engine	Completed	Hash table construction, posting node generation, dynamic insertion logic.
Module 3: Query Parser & Operations	Completed	C++ string parsing, two-pointer linear AND/OR/NOT algorithms, unit tests.
Module 4: TF-IDF Ranking & CLI	In Progress	TF calculation completed; IDF weighting and Min-Heap sorting in development.

Overall Project Completion: 30%

Working Prototype / Demonstration

Implemented Features

- Fast string tokenization and lowercase normalization
- Linked list posting generation with Term Frequencies
- $O(M + N)$ Linear Boolean Set Operations (AND, OR, NOT)
- Memory-leak safe dynamic heap management

Prototype / Output

```
$ ./test_runner
Test 1:  Tokens -> [malloc] [and] [pointer]
Test 2:  Boolean AND -> [Doc 3] -> [Doc 5] ->
NULL
Test 3:  Boolean OR -> [Doc 1, 2, 3, 5, 8, 10]
Test 4:  Boolean NOT -> [Doc 1] -> [Doc 8] ->
NULL

>> All tests executed!  Memory released
cleanly.
```

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Query Tokenizer	Lowercased tokens without delimiters	Tokens extracted cleanly into vector	Pass
Boolean AND	Intersection of posting lists	Returned common Doc IDs in sorted order	Pass
Boolean OR	Union without duplicate Doc IDs	Combined Doc IDs with summed TF	Pass
Boolean NOT	Set difference ($P_1 \setminus P_2$)	Excluded all occurrences of second term	Pass

Results / Validation

Linear two-pointer operations verified with $O(M + N)$ complexity. Valgrind testing confirmed zero memory leaks after executing `free_result_postings()`.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: github.com/Team/CodeSearchEngine
- README.md: Updated with build instructions
- Contributors: All 4 Members Added
- Repository Status: Actively Maintained
- Branches: `main`, `feature/query-parser`

Development Activity

- Regular modular commits across all components
- Descriptive commit messages following conventions
- Individual contribution verifiable across features
- Modular directory structure (`include/`, `src/`)
- Technical documentation and test cases updated

Requirement: All team members must be contributors and must **commit individually using their own GitHub accounts**.

Individual Contribution

Member	Role	Technical Contribution	GitHub-Commits
Anmol Gupta	Ingestion Pipeline and Lexical Tokenizer (C++)	Implemented source code ingestion pipeline, comment filtering, and file scanner.	24
Harshit Kumar	Query Parsing and Boolean Operations (Pure C / C++)	Built C++ query tokenizer, two-pointer Boolean retrieval (AND, OR, NOT), and memory deallocation.	7
Atharva Goyal	Relevance Ranking, Min-Heap and CLI Interface (Pure C / C++)	Engineered in-memory inverted index using djb2 hash table and sorted linked lists.	6
Navneet Nainwal	In-Memory Inverted Index Engine (Pure C)	Integrated CLI front-end, Term Frequency scoring, and initial Min-Heap ranker.	4

Individual Evaluation

Every member should be prepared to explain and demonstrate their own technical contribution to the project.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- **C/C++ Interoperability:** Resolving name mangling via `extern "C"`.
- **Memory Management:** Handling pointer manipulation during set union and intersection.

Solutions / Learning

- Engineered centralized `free_result_postings()` memory lifecycle.
- Decoupled modular headers for zero cross-file compile issues.

Phase-III Roadmap

1. Complete TF-IDF ranking module
2. Integrate Min-Heap top-K results retrieval
3. Implement code snippet highlighting in CLI
4. Benchmark search latency against naive linear scans
5. Finalize comprehensive project report

Expected Outcomes & References

Expected Final Outcomes

- Fully functional, high-speed CLI code search engine
- Sub-millisecond retrieval on multi-file codebases
- Scalable inverted index architecture with low memory footprint
- Practical developer utility for local code searching

References

1. Manning et al., *Introduction to Information Retrieval*, Cambridge Univ. Press.
2. Kernighan & Ritchie, *The C Programming Language*, 2nd Ed.
3. Stroustrup, *The C++ Programming Language*, 4th Ed.
4. Cormen et al., *Introduction to Algorithms* (CLRS), MIT Press.

Thank You — Questions & Suggestions